Operating Systems - Lecture Notes

Processes

A process is a program in execution, together with its current state.

Each process has its own address space containing code, data, a heap and a stack.

The process control block (PCB) stores the information the operating system needs to manage a process, such as its program counter, registers and scheduling priority.

A process moves between the new, ready, running, waiting and terminated states during its lifetime.

Threads

A thread is the smallest unit of execution that can be scheduled by the operating system.

Threads within the same process share the address space but each thread has its own stack and registers.

Creating a thread is cheaper than creating a process because no new address space has to be set up.

A context switch saves the state of the running thread and restores the state of the next one.

CPU Scheduling

The CPU scheduler decides which ready process runs next on the processor.

First-Come, First-Served (FCFS) scheduling runs processes in the order they arrive and can cause the convoy effect.

Shortest Job First (SJF) scheduling minimises the average waiting time but requires knowing burst lengths in advance.

Round Robin scheduling gives each process a fixed time quantum, typically between 10 and 100 milliseconds.

Preemptive scheduling allows the operating system to interrupt a running process when a higher-priority process becomes ready.

Synchronization

A race condition occurs when the result of a computation depends on the timing of concurrent threads.

A critical section is a part of the code that accesses shared data and must not be executed by more than one thread at a time.

A mutex is a lock that provides mutual exclusion for a critical section.

A semaphore is an integer variable that is accessed only through the atomic operations wait and signal.

Deadlocks

A deadlock is a situation in which a set of processes are each waiting for a resource held by another process in the set.

The four Coffman conditions for deadlock are mutual exclusion, hold and wait, no preemption and circular wait.

The Banker's algorithm avoids deadlock by only granting a request if the system remains in a safe state.

Deadlock prevention works by making sure that at least one of the four necessary conditions can never hold.